WORKING BASELINE

AI Project Memory

V0.1 开发变更与学习指南

多 AI 会话导入 · 跨项目检索 · 逐句可追溯 · 本地数据库

01	统一导入	ChatGPT + 通用 AI JSON
02	事实底座	项目 / 对话 / 消息 / 句子
03	全局查询	问题目录 + 跨项目检索
04	证据坐标	原句 ID + 字符起止位置

版本 0.1.0 | 2026-09-13 | 本地最小可运行数据闭环

目录

1. 这次真正做成了什么 3
2. 对产品定位的重新审核 3
3. 本次代码与文件变更 4
4. 数据如何流动 4
5. 你现在可以怎样运行 4
6. 当前 API 调用方式 5
7. 已完成的验证 5
8. 仍然没有做的部分 5
9. 如果由你亲自开发，建议这样学习 6
   - 9.1 第一阶段：先理解数据，不急着学 AI . . . 6
   - 9.2 第二阶段：学最少量 SQLite . . . 6
   - 9.3 第三阶段：理解 Python 分层 . . . 6
   - 9.4 第四阶段：理解 HTTP API . . . 6
   - 9.5 第五阶段：再进入 LLM 结构化抽取 . . . 6
10. 建议的下一版本：V0.2 知识编译最小闭环 6
11. 对项目价值的判断 7
12. 本版本完成定义 7

1. 这次真正做成了什么

V0.1 已经把最重要、也最容易被 UI 掩盖的底座跑通：把不同 AI 来源的会话转换成统一结构，写入本地 SQLite 数据库，逐句建立可追溯索引，识别显式问题及其直接回答，并允许跨项目检索。它还不是完整产品，但已经是一条可验证的数据闭环，而不只是文件夹骨架或概念方案。

用户现在可以完成以下闭环：导入“学习”项目的多条对话 -> 查看曾经问过的问题 -> 搜索“复习”等主题 -> 得到来自不同对话甚至不同项目的命中 -> 根据项目、对话、消息、句子起止位置返回原文。

能力	V0.1 状态	实际含义
多 AI 兼容入口	已实现	通用 JSON 适配器可承接 Claude、Gemini、本地模型等后续转换结果
ChatGPT 导出	已实现	可解析 conversations.json 的会话映射与父子消息关系
本地数据库	已实现	无需安装数据库服务器，默认使用 SQLite
问题/直接回答目录	已实现	保存问题句，并关联紧随其后的回答消息
跨项目检索	已实现	可从所有已授权导入的项目中检索句子
逐句溯源	已实现	每个命中包含项目、对话、消息 ID 与字符起止位置
HTTP API	已实现	提供项目、问题、搜索和消息读取接口
AI 归纳文件树	尚未实现	计划从 V0.2 开始加入主题抽取、证据绑定与 Markdown 生成

2. 对产品定位的重新审核

项目目录虽然叫 GPT-SaaS，但产品应定位为“AI Project Memory / AI 项目记忆与知识编译层”。GPT、ChatGPT 只是第一个连接器，不是产品边界。真正的核心对象不是某家模型，而是用户拥有或被授权处理的项目、对话、消息、句子、问题、回答、主题和知识文件。

“跨对话、跨项目权限”不能理解成绕过平台权限读取任意聊天。正确做法是：用户主动导出、上传或连接自己有权访问的来源；系统把这些来源复制到自己的受控索引；检索与 AI 归纳只在用户授权范围内执行；每条新结论保存证据链接。这样才能同时满足可用性、隐私和可追溯性。

V0.1 因而遵循四项原则：供应商无关、原文优先、结论可追溯、权限先于智能。未来即使接入更强的模型或向量数据库，这四项原则也不变。

3. 本次代码与文件变更

位置	用途	你应该先理解什么
ai_memory/connectors/	把不同供应商数据转换成统一会话结构	适配器模式、JSON 字段映射
ai_memory/database.py	创建 SQLite 表、索引和版本号	主键、外键、唯一约束、索引
ai_memory/service.py	导入、去重、问题关联、跨项目搜索	业务逻辑和数据库逻辑的分工
ai_memory/text.py	按句切分并保存字符位置	为什么摘要必须能回到原文
ai_memory/api.py	暴露只读本地 HTTP 接口	URL、查询参数、JSON 响应
ai_memory/cli.py	提供初始化、导入、查询和启动服务命令	命令行参数与程序入口
schemas/generic-import.v0.1.schema.json	定义通用导入格式	Schema 是数据的“合同”
contracts/openapi-v0.1.yaml	记录当前实际可用接口	前后端如何通过接口契约协作
tests/test_v01.py	验证关键行为	测试不是形式，而是防止以后改坏

4. 数据如何流动

一次导入会依次经过五层：来源文件 -> 连接器 -> 统一会话模型 -> SQLite 关系表 -> CLI/API 查询。连接器只负责“读懂外部格式”，服务层负责稳定 ID、去重、切句和关联，数据库层负责持久化，API 层只负责对外读取。

核心数据关系如下：一个项目包含多条对话；一条对话包含多条消息；一条消息被切成多个句子；用户消息中的问号句会形成问题；回答消息通过父消息关系连接到问题。检索结果落在句子层，而不是只返回一段脱离上下文的摘要。

数据表	保存内容	关键设计
projects	来源、外部项目 ID、标题	同一来源与外部 ID 唯一
conversations	会话标题、时间与所属项目	保留原始会话边界
messages	角色、正文、父消息、哈希	原文是事实层，不被摘要覆盖
sentence_segments	句子正文、起止字符、哈希	搜索结果可精确定位回消息
questions	问题句、状态、来源句子	形成“我问过什么”目录
question_answers	问题与回答消息的关系	V0.1 只做直接父子回答
import_runs	文件哈希、来源路径、导入结果	同一文件重复导入会跳过

5. 你现在可以怎样运行

在项目根目录打开 PowerShell。使用 Python 3.11 或更高版本，不需要先安装 PostgreSQL、Docker 或云服务。

```
python -m ai_memory init
python -m ai_memory import examples/learning-project.json
python -m ai_memory projects
python -m ai_memory questions
python -m ai_memory search 复习
python -m ai_memory serve
```

默认数据库位于 data/ai-memory-v0.1.sqlite。如果想做一次不会影响正式数据的实验，可以把 --db tmp/my-test.sqlite 放在子命令之前。再次导入相同内容时，程序会根据文件 SHA-256 哈希返回 skipped，避

免数据重复。

通用导入文件的最小结构如下。任何 AI 平台只要能转换成这套结构，就可以进入同一个数据库。

```
{
  "provider": "any-llm",
  "project_id": "learning",
  "project_title": "学习",
  "conversations": [{
    "id": "conversation-1",
    "title": "学习方法",
    "messages": [
      {"id": "u1", "role": "user", "text": "如何安排复习？"},
      {"id": "a1", "parent_id": "u1", "role": "assistant", "text": "先主动回忆，再检查。"}
    ]
  }]
}
```

6. 当前 API 调用方式

启动服务后，本地地址默认为 http://127.0.0.1:8765。V0.1 API 刻意保持只读，导入仍通过命令行完成，这能减少早期数据被误改的风险。

请求	作用
GET /health	检查服务与版本
GET /v0/projects	查看所有已导入项目
GET /v0/questions?limit=100	查看跨项目问题目录
GET /v0/projects/{id}/conversations	查看一个项目的会话
GET /v0/projects/{id}/questions	查看一个项目的问题
GET /v0/search?q=复习	跨项目搜索句子
GET /v0/search?q=复习&project_id={id}	限制在一个项目内搜索
GET /v0/messages/{id}	根据命中回看整条原始消息

搜索返回的核心不是一句“AI 说了什么”，而是 project_id + conversation_id + message_id + sentence_id + start_offset + end_offset。这些字段共同构成证据坐标，未来生成知识文件时也必须保留。

7. 已完成的验证

本版本运行了 4 项自动测试，全部通过：中文句子切分后能按起止位置还原原文；同一文件第二次导入会跳过；问题能关联直接回答；ChatGPT 导出中的父节点 ID 能映射到真实消息 ID；跨项目搜索能同时返回“学习”和“高数”等不同项目命中。

另外启动了实际 HTTP 服务并调用健康检查与搜索接口：服务返回版本 0.1.0，示例“学习”项目中搜索“复习”得到 4 条句子级命中，每条均含字符位置。代码也通过 Python 编译检查。

这些测试证明数据闭环已经成立，但不代表产品已经成熟。V0.1 没有验证海量数据性能、复杂分支会话、所有 ChatGPT 导出变体、权限隔离、内容删除同步和模型归纳质量。

8. 仍然没有做的部分

V0.1 不调用 LLM 自动总结，也不生成“学习方法/主动回忆.md”这样的知识文件。原因不是这部分不重要，而是它必须建立在稳定事实层之上。若先让 AI 总结、后补来源，内容很容易失去证据、重复或互相矛盾。

以下能力进入后续版本：主题提取与跨对话聚类；一个句子被多个主题引用；知识文件树生成；核心总结与 AI 扩展内容分区；证据脚注；人工确认与驳回；增量更新；观点演化时间线；语义向量搜索；真实账户连接器；多用户权限、删除和审计。

V0.1 的问题识别目前依赖问号，回答关联主要依赖直接父子消息。这是明确的基线，不应被包装成“已经理解所有问题”。V0.2 可用 LLM 结构化输出增强识别，但仍需保留规则结果和原始证据。

9. 如果由你亲自开发，建议这样学习

9.1 第一阶段：先理解数据，不急着学 AI

目标是能读懂 `examples/learning-project.json`，知道对象、数组、字符串、ID 和父子关系分别是什么。练习：手工增加一条“专业课”对话并导入，再用 `questions` 和 `search` 查看变化。

9.2 第二阶段：学最少量 SQLite

先掌握表、行、列、主键、外键、唯一约束、索引和 `SELECT`。练习：使用 SQLite 查看 `projects`、`messages`、`sentence_segments` 三张表，并解释为什么句子不能只保存文本而不保存位置。

9.3 第三阶段：理解 Python 分层

按 `connector -> service -> database -> api` 顺序阅读。连接器解决格式差异，服务层表达产品规则，数据库保存结果，API 给 UI 使用。练习：复制通用连接器，做一个你自己的模拟 AI 格式适配器。

9.4 第四阶段：理解 HTTP API

学习 GET、路径、查询参数、状态码和 JSON。练习：启动服务，在浏览器打开 `/health` 和 `/v0/search?q=复习`，再根据返回的 `message_id` 请求原消息。

9.5 第五阶段：再进入 LLM 结构化抽取

先学 Prompt、JSON Schema、温度、重试、令牌成本和验证。模型输出只能成为“候选知识”，不能覆盖原始消息。练习：让模型为 10 个句子返回主题标签、摘要和引用句子 ID，再人工检查是否每条摘要都有证据。

学习模块	达标标准	建议先不学的内容
JSON	能手改示例并成功导入	复杂前端状态管理
SQLite	能解释 7 张表的关系	数据库集群和分库分表
Python	能跟踪一次导入调用链	大型框架内部原理
HTTP	能独立调用 3 个接口	微服务和 Kubernetes
LLM 抽取	能验证结构化结果和引用	多 Agent 自主规划

10. 建议的下一版本：V0.2 知识编译最小闭环

V0.2 不宜先做漂亮 UI，也不宜立刻接入所有平台。最有价值的下一步是把已入库句子编译成可追溯的 Markdown 知识文件，并允许人确认。

建议顺序：第一，增加 `topics`、`topic_evidence`、`knowledge_documents` 和 `document_revisions`；第二，用可替换的 LLM 接口抽取“主题候选、核心观点、证据句子 ID”；第三，把同一主题跨对话归并；第四，生成“核

心内容、AI 扩展、来源与冲突”四段式 Markdown；第五，增加人工确认状态；第六，为新增会话做局部重编译，而不是全库重算。

V0.2 的验收标准应是：从“学习方法”和“专业课”两条混杂对话中找出所有通用学习方法；生成一个结构清晰的 学习方法/ 文件树；每条核心观点至少指向一个原句；AI 扩展与原始归纳视觉上分开；删除来源后能标记受影响知识，而不是悄悄保留。

11. 对项目价值的判断

这个方向有真实价值，因为现有 AI 聊天的主要问题往往不是“没有回答”，而是回答散落、重复、跨主题、难以回看，也很难知道某个总结来自哪一句。把聊天记录变成可追溯、可重编译的个人知识工程，和普通聊天搜索、普通笔记、单次总结都有明显区别。

它是否会在 GitHub 上受欢迎，取决于三个条件：导入过程足够简单；生成结果可信且能回到原文；仓库有清晰示例、隐私说明和逐版本可运行成果。V0.1 已经开始解决第二项，并为第一项建立接口。比“功能很多但无法验证”更重要的是持续交付小而完整的版本。

12. 本版本完成定义

V0.1 的完成标准是“能够把授权的多会话数据安全地放进一个统一、可查询、可追溯的本地事实库”。这个标准已经达到。当前最合理的推进方式是先由你实际运行示例、理解数据结构，再进入 V0.2 的 AI 知识编译，而不是现在就把它包装成完成品。